

GENERATORS AND OBJECTS

COMPUTER SCIENCE MENTORS 61A

October 14 – October 17, 2025

1 Generators

1. Write a generator function that takes in an iterator `it` and yields the running total of the elements produced by it.

```
def accumulate(it):  
    '''  
    >>> def all_ints():  
    ...     i = 0  
    ...     while True:  
    ...         yield i  
    ...         i += 1  
    >>> a = accumulate(all_ints())  
    >>> [next(a) for x in range(6)]  
    [0, 1, 3, 6, 10, 15]  
    '''
```

2. Define `all_sums`, a generator that iterates through all the possible sums of elements from `lst`. (Repeat sums are permitted.)

```
def all_sums(lst):  
    '''  
    >>> list(all_sums([]))  
    [0]  
    >>> list(all_sums([1, 2]))  
    [3, 2, 1, 0]  
    >>> list(all_sums([1, 2, 3]))  
    [6, 5, 4, 3, 3, 2, 1, 0]  
    >>> list(all_sums([1, 2, 7]))  
    [10, 9, 8, 7, 3, 2, 1, 0]  
    '''
```

3. Write a generator that, given `m` (the amount of money you have), `pc` (the cost of one pear), and `ac` (the cost of one apple) yields all possible combinations of fruit that you can buy that uses up the most of your money. In other words, each combination of fruits should not result in enough money left over to buy another fruit. Combinations of the same number of each fruit in different orders is okay. It is also okay if each combination has an extra space at the end.

```
def fruitOptions(m, pc, ac):  
    '''  
    >>> print(list(fruitOptions(10, 2, 5)))  
    ['pear pear pear pear pear ', 'pear pear apple ', 'pear apple pear ',  
    'apple pear pear ', 'apple apple ']  
    '''  
    if _____:  
        yield ''  
    if m >= pc:  
        for _____:  
            _____  
    if m >= ac:  
        for _____:  
            _____
```

2 Objects and Attributes

1. Let's build a Bear using OOP!

Bear instances should have an attribute `name` that holds the name of the bear. The Bear class should have an attribute `bears`, a list that stores the name of each bear.

```
>>> oski = Bear('Oski')
>>> oski.name
'Oski'
>>> Bear.bears
['Oski']
>>> winnie = Bear('Winnie')
>>> Bear.bears
['Oski', 'Winnie']
```

```
class Bear:
```

2. Implement the classes so the following code runs.

```
'''
>>> p = Plant()
>>> p.height
1
>>> p.materials
[]
>>> p.absorb()
>>> p.materials
[|Sugar|]
>>> Sugar.sugars_created
1
>>> p.leaf.sugars_used
0
>>> p.grow()
>>> p.materials
[]
>>> p.height
2
>>> p.leaf.sugars_used
1
'''
```

```
class Plant:
    def __init__(self):
        '''A Plant has a Leaf, a list of sugars created so far,
```

```

        and an initial height of 1.
        '''
        self.leaf = Leaf(self)
        self.materials = _____
        self.height = _____

    def absorb(self):
        '''Calls the Leaf to create sugar.'''

    def grow(self):
        '''A Plant consumes all of its sugars to grow, each of which
        increases its height by 1.
        '''

class Leaf:
    def __init__(self, plant): # plant is a Plant instance
        '''A Leaf is initially alive, and keeps track of how many
        sugars it has created.
        '''

    def absorb(self):
        '''If this Leaf is alive, a Sugar is added to the plant's
        list of sugars.
        '''
        if self.alive:

    def __repr__(self):
        return '|Leaf|'

class Sugar:
    sugars_created = 0

    def __init__(self, leaf, plant):

    def activate(self):
        '''A sugar is used.'''

```

```
def __repr__(self):
    return '|Sugar|'
```

3. **Musician** What would Python display? Write the result of executing the code and the prompts below. If a function is returned, write 'Function'. If nothing is returned, write 'Nothing'. If an error occurs, write 'Error'.

```
class Musician:
    popularity = 0
    def __init__(self, instrument):
        self.instrument = instrument
    def perform(self):
        print('a stellar ' + self.instrument + ' performance')
        self.popularity = self.popularity + 2
    def __repr__(self):
        return self.instrument

class BandLeader(Musician):
    def __init__(self):
        self.band = []
    def recruit(self, musician):
        self.band.append(musician)
    def perform(self, song):
        for m in self.band:
            m.perform()
        Musician.popularity += 1
        print(song)
    def __str__(self):
        return 'Here's the band!'
    def __repr__(self):
        band = ''
        for m in self.band:
            band += str(m) + ' '
        return band[:-1]

miles = Musician('trumpet')
goodman = Musician('clarinet')
ellington = BandLeader()
```

```
>>> ellington.recruit(goodman)
>>> ellington.perform()
```

```
>>> ellington.perform('sing, sing, sing')
```

```
>>> goodman.popularity, miles.popularity
```

```
>>> ellington.recruit(miles)
>>> ellington.perform('caravan')
```

```
>>> ellington.popularity, goodman.popularity, miles.popularity
```

```
>>> print(ellington)
```

```
>>> ellington
```